

Multi-hop Evidence Retrieval

1 任务说明

多跳问答 (Multi-hop Question Answering) 是指回答一个问题需要从多个文档中检索并综合多条证据。本次实验使用 **MuSiQue** (Multihop Questions via Single-hop Question Composition) 数据集, 要求从一个大规模段落语料库中, 针对给定的多跳问题检索出相关的支持性段落 (supporting paragraphs)。你可以参考文档最后列出的资料与资源以获得更多相关知识。本次作业的截止日期为 **5 月 29 日 23: 59**。

2 数据集

本实验使用 MuSiQue 数据集 (dgslibisey/MuSiQue, HuggingFace)。我们从验证集中构建了:

1. **语料库** (data/corpus.jsonl): 对验证集中所有问题的支持性段落去重得到, 共约 21,100 个段落。每条数据格式如下:

```
{  
    "corpus_id" : <段落编号>,  
    "title": <段落标题>,  
    "text": <段落内容>  
}
```

2. **测试集**: 按照问题的推理跳数 (hop count), 从验证集中分别采样 200 条, 共三个测试集:

- (a) data/test_2hop.jsonl : 200 条 2-hop 问题
- (b) data/test_3hop.jsonl : 200 条 3-hop 问题
- (c) data/test_4hop.jsonl : 200 条 4-hop 问题

每条数据格式如下:

```
{  
    "id" : <测试数据编号>,  
    "question": <段落标题>,  
    "gold_corpus_ids": <所有标准支持性段落对应的语料库段落编号组成的列表>  
}
```

你需要分别实现稀疏检索和稠密检索，并在三个测试集上进行测试。给定一个测试集中的问题 (question)，你需要从语料库 (21,100 个段落) 中检索出 **Top-5** 相关段落。评估指标为 **Recall@5**，即对每个问题，计算 Top-5 检索结果中包含的标准支持性段落占全部标准支持性段落的比例，然后对所有问题取平均。

$$\text{Recall@5} = \frac{1}{|Q|} \sum_{q \in Q} \frac{|\text{Retrieved}_5(q) \cap \text{Gold}(q)|}{|\text{Gold}(q)|}$$

数据准备脚本 (scripts/prepare_data.py) 和评估脚本 (scripts/evaluate.py) 已提供。

3 任务内容

3.1 BM25 稀疏检索 (30%)

实现基于 BM25 的稀疏检索方法，从全局语料库中检索 Top-5 段落。

要求：

- 可以使用现有的 BM25 库 (如 rank_bm25)。
- 对三个测试集分别进行检索并评估 Recall@5。
- 在报告中介绍 BM25 的原理、你的实现细节。

3.2 Dense Retrieval 稠密检索 (40%)

实现基于预训练语言模型的稠密检索方法，从全局语料库中检索 Top-5 段落。

要求：

- 使用预训练的文本嵌入模型（如 BGE、Qwen-Embedding 等）对语料库段落和查询进行向量编码。
- 对三个测试集分别进行检索并评估 Recall@5。
- 在报告中介绍你选用的模型及实现细节。

提示：

- 推荐先用 `scripts/build_index.py` 对语料库进行一次性编码并构建 FAISS 索引，然后在检索时直接加载索引。
- 对查询端可使用模型提供的 `instruction/prompt` 模板以提升检索效果。
- 该部分推荐使用 GPU 完成，计算资源有限的同学可以选择较小的模型（如 0.6B 级别的嵌入模型）。

3.1 和 3.2 中需要将每个数据集上的预测结果存储在一个 JSONL 文件中，文件的每一行是一个 JSON 对象，格式为：

```
{  
  "id" : <测试数据编号>,  
  "retrieved_corpus_ids": <预测的Top5段落编号组成的列表>,  
}
```

使用评估脚本验证你的结果：

```
python scripts/evaluate.py --prediction <你的预测文件> --gold data/test_2hop.jsonl
```

3.3 评估与分析 (30%)

对 BM25 和 Dense Retrieval 的结果进行全面的对比分析。

要求：

- 分析不同跳数下 BM25 和 Dense Retrieval 的性能差异及原因。
- 给出具体的检索案例分析 (case study)：展示若干 BM25 检索成功但 Dense 失败的案例，以及 Dense 检索成功但 BM25 失败的案例，分析原因。

4 评分标准

4.1 实现分 (50%)

你只需顺利实现 BM25 和 Dense 检索即可获得该部分的全部分数。

部分	分值	内容
3.1 BM25	20%	BM25 实现及结果
3.2 Dense Retrieval	30%	Dense 检索实现及结果

4.2 Recall@5 指标分 (30%)

你实现的稀疏检索和稠密检索在三个测试集上的 **平均 Recall@5** 将与以下 Baseline 进行对比评分：

方法	2-hop	3-hop	4-hop	分值
稀疏检索	0.37	0.27	0.15	10%
稠密检索	0.60	0.53	0.30	20%

评分规则：

- 平均 Recall@5 达到 Baseline 即可获得该部分全部分数。
- 平均 Recall@5 低于 Baseline 进行线性计分。

4.3 报告分 (20%)

你需要提交一份实验报告，包括以下内容：

1. 基于 BM25 的稀疏检索方法介绍和实现细节，以及在三个测试集上的 Recall@5 结果
2. 基于预训练语言模型的稠密检索方法介绍和实现细节，以及在三个测试集上的 Recall@5 结果
3. 对两种方法性能差异的分析和具体案例分析

5 提交要求与迟交政策

通过课程网站以 **zip** 压缩包形式提交作业，压缩包中应包含：

- 一份 **PDF** 格式的实验报告（使用作业文件夹中提供的 LaTeX 模板生成）。
- 源代码（Python），附带 **README** 文件，描述依赖、代码结构和运行方式，无需提交数据集和模型权重。
- 预测结果文件（**JSONL** 格式）。

迟交的作业每晚交一天，将从最终成绩中扣除 5%。在截止日期一周后（6 月 5 日 23:59），将不再接受任何作业提交。你可以无限次地重新提交，你的作业提交时间将以你最后一次在课程系统中提交的时间为准。因此，除非你确信新的提交内容有了显著改进，足以弥补因迟交而扣除的分数，否则请不要在截止日期当天的午夜之后重新提交。提交时间没有任何商量的余地，因迟交而扣除的分数将四舍五入至最接近的整数。

6 运行指南

安装依赖

```
pip install -r requirements.txt
```

1. 下载数据集并构建测试集（已提供，无需重新运行）

```
python scripts/prepare_data.py
```

2. 构建 FAISS 向量索引（用于 Dense Retrieval）

```
python scripts/build_index.py --batch-size 64 --device cuda
```

3. 评估

```
python scripts/evaluate.py --prediction outputs/bm25/predictions_2hop.jsonl  
--gold data/test_2hop.jsonl
```

```
python scripts/evaluate.py --prediction outputs/dense/predictions_2hop.jsonl  
--gold data/test_2hop.jsonl
```

7 联系助教

- luokangcheng@stu.pku.edu.cn (罗康诚)
- jiayi_li@stu.pku.edu.cn (李佳一)

8 参考资料与资源

1. Robertson, S. E., Walker, S., Jones, S., Hancock-Beaulieu, M. M., & Gatford, M. (1995). Okapi at TREC-3. *Nist Special Publication Sp*, 109, 109.
2. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP 2020*.
3. Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2022). MuSiQue: Multihop Questions via Single-hop Question Composition. *TACL 2022*.
4. [rank_bm25](https://github.com/dorianbrown/rank_bm25) —BM25 算法的 Python 实现
5. [FAISS](<https://github.com/facebookresearch/faiss>) —高效向量相似度搜索库
6. [Sentence-Transformers](<https://www.sbert.net/>) —预训练文本嵌入模型框架
7. [Qwen3-Embedding](<https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>) —Qwen3 系列文本嵌入模型
8. [BGE Embedding Models](<https://huggingface.co/BAAI/bge-large-en-v1.5>) —BAAI 通用嵌入模型